1. Initialization — the beginning of the waveform

Signals to watch: clk, reset, start, tx_inst/state

At the beginning, the clock starts toggling, creating the timing reference for the entire design. Your clock period is 10 ns, so a rising edge occurs every 10 ns.

Initially, reset is high. Both the transmitter and receiver use this active-high reset to initialize their state machines and internal registers. After reset is deasserted, the system is ready to operate.

The transmitter and receiver initially remain in IDLE. This means neither controller has started its processing sequence yet. When start pulses high, the transmitter detects the request on a rising clock edge and begins its transmission process.

→ "The waveform begins with clock generation and reset initialization. After reset is released, the transmitter waits in the idle state. The start pulse initiates the encoding and transmission sequence."

2. RS encoding — generating eight codewords

Signals to watch: tx_inst/state, tx_inst/rs_start, tx_inst/rs_inst/busy_i, tx_inst/rs_inst/done_i, tx_inst/code_index

Your original input contains 15,296 bits, which are divided into eight blocks of 1,912 bits each. Each block contains 239 bytes.

When the transmitter enters START_RS, it requests the RS encoder to process the current block. The controller then enters WAIT_RS, where it waits for the encoder to finish.

Your transmitter has 8 blocks, indexed from 0 to 7.

- **code_index = 0 → first block**
- **code_index = 1 → second block**
- **…**
- **code_index = 7 → eighth and final block**

Each block contains 239 bytes (1,912 bits) of original data. The RS encoder adds 16 parity bytes, producing a 255-byte codeword.

During encoding, busy_i indicates that the RS encoder is working. Once encoding finishes, done_i signals completion, and the transmitter stores the resulting codeword.

Each RS codeword contains 255 bytes: 239 data bytes and 16 parity bytes. The parity bytes provide redundancy that the receiver can use for error correction.

This sequence repeats for all eight blocks. The code_index progresses from 0 through 7. After the eighth codeword has been collected, the transmitter moves to PREPARE.

What to look for on the waveform: repeated START_RS and WAIT_RS states, the encoder's busy and done activity, and the progression of code_index.

→ "The transmitter processes the input in eight blocks. Each block is encoded by the RS encoder, which adds 16 parity bytes to the 239 data bytes. The transmitter collects all eight completed codewords before starting interleaving."

3. Interleaving — rearranging the RS symbols

Signals to watch: tx_inst/state, tx_inst/symbol_index, tx_inst/codeword_select, tx_inst/interleaved

After all eight codewords have been collected, the transmitter enters PREPARE. This is where depth-8 interleaving takes place.

The controller selects an 8-bit symbol from one RS codeword at a time. It first processes symbol position 0 across codewords 0 through 7. It then advances to symbol position 1 and repeats the process.

This continues across all 255 symbol positions. The resulting stream contains 16,320 bits, arranged in the interleaved order.

Why is this necessary? If a burst of errors affects consecutive transmitted symbols, interleaving can distribute those errors across multiple RS codewords. This can make the errors easier for the RS decoder to correct.

The interleaved signal is a very wide bus, so the waveform viewer may not display its changing contents clearly at the full simulation zoom. If it shows "No Data," we cannot conclude that the interleaving worked or failed from that row alone.

What to look for: symbol_index advancing through 254, codeword_select cycling through 0–7, and the controller eventually moving from PREPARE to PREPARE_WAIT, then SEND_BITS.

→ “After RS encoding, the transmitter rearranges symbols from eight codewords using depth-8 interleaving. This spreads burst errors across codewords. After all symbols have been rearranged, the transmitter starts convolutional encoding.”

4. Convolutional encoding — generating the transmitted bitstream

Signals to watch: tx_inst/state, tx_inst/bit_index, tx_inst/cc_data, tx_inst/cc_valid, tx_inst/cc_inst/code_out, tx_inst/cc_valid_out, tx_inst/cc_count

1. state = SEND_BITS — The transmitter has entered the CC encoding stage and is feeding the interleaved bits to the encoder.

2. bit_index — It advances from 0 to 1, 2, 3, and onward. This tracks the input bits being processed.

3. cc_data — This signal changes between 0 and 1 according to the interleaved input data. Each value is an input bit for the CC encoder.

4. cc_valid — It goes HIGH when the transmitter presents valid input data to the encoder.

5. cc_code — You can see values such as 1, 2, 0, and 3. This is a 2-bit signal, so these hexadecimal-looking values represent the encoded output pairs: 1 = 01, 2 = 10, 0 = 00, and 3 = 11.

6. cc_valid_out — It indicates when the encoded output pair is valid. The transmitter uses this signal to know when to collect a pair.

7. cc_count — It increases as the output pairs are collected. In your screenshot, it is only around the first few dozen pairs, because you are zoomed into the beginning of the encoding stage.

8. tx_bits(100) — This particular bit of the final transmitted vector is currently 0. It represents just one position in the complete transmitted stream, not the entire stream.

5. Transmission completion

Signals to watch: tx_inst/state, tx_done, tx_bits

Once all 16,320 encoded output pairs have been collected, the transmitter completes its operation. The controller briefly enters COMPLETE, deasserts its busy status, asserts its done status, and returns to IDLE.

The completed stream is available at tx_bits.

Your full-chain simulation reported transmitter completion at approximately 183,945 ns. This is the end of the transmitter stage, not the end of the entire communication chain.

What to tell your guide:

“The transmitter completes the RS encoding, interleaving, and convolutional encoding stages and produces the final coded stream for transmission.”

6. Channel error injection — deliberately corrupting one bit

Signals to watch: tx_bits(100), channel_bits(100)

Immediately after transmitter completion, your testbench copies the transmitted stream into the channel stream and deliberately flips bit 100.

This is a controlled test condition. The bit is inverted: a transmitted zero becomes a one, or a transmitted one becomes a zero. The other bits are left unchanged by this particular error-injection operation.

The testbench reports the error injection at approximately 183,946 ns, one nanosecond after the transmitter-completion report.

On the waveform, compare the two individual-bit traces. At the error position, the channel bit should be opposite to the transmitted bit.

What to tell your guide:

“I deliberately introduced a single-bit error in the channel to test whether the receiver can recover the original data from a corrupted transmitted stream.”

Receiver: start to finish

Stage 1 — Receiver starts

Signals: rx_start, rx_inst/state

- Initially, rx_inst/state = IDLE. The receiver is waiting for data to be processed.
- After transmission finishes and the channel error is injected, rx_start pulses HIGH.
- The receiver detects this signal and changes its state from IDLE to WAIT_VITERBI.

Meaning: The receiver has been told to begin recovering the original data.

Stage 2 — Viterbi decoding starts

Signals: rx_inst/vit_start, rx_inst/vit_inst/state, rx_inst/vit_inst/busy, rx_inst/vit_inst/step

- vit_start pulses HIGH to request Viterbi decoding.
- The internal Viterbi state changes from IDLE to CALCULATE.
- busy goes HIGH while the decoder processes the received coded bits.
- step increases as the decoder calculates the most likely path through the convolutional code.

Meaning: The receiver is trying to reconstruct the original bit sequence from the coded data, which may contain a transmission error.

Stage 3 — Viterbi traceback and completion

Signals: rx_inst/vit_inst/state, rx_inst/vit_inst/trace_step, rx_inst/vit_inst/busy, rx_inst/vit_done

- The Viterbi state changes from CALCULATE to TRACEBACK.
- trace_step counts backwards through the stored decisions, from 8 toward 1 in your displayed waveform.
- The state then changes to FINISH and returns to IDLE.
- busy goes LOW, and vit_done pulses HIGH.

Meaning: The Viterbi decoder has completed its work and produced the decoded bit sequence.

Stage 4 — Receiver starts RS decoding

Signals: rx_inst/state, rx_inst/rs_start, rx_inst/rs_busy

- The receiver detects vit_done and moves from WAIT_VITERBI to WAIT_RS.
- rs_start pulses HIGH to start the eight RS decoder instances.
- rs_busy is an 8-bit vector. In your waveform, FF means all eight bits are HIGH.

Meaning: The receiver is now processing the eight RS codewords to detect and correct errors using their parity information.

Stage 5 — RS decoders finish

Signals: rx_inst/rs_busy, rx_inst/rs_done, rx_inst/rs_error, rx_inst/rs_valid

- Each RS decoder processes its codeword.
- rs_busy indicates decoder activity, while rs_done indicates which decoders have completed.
- The receiver waits until rs_done = FF, meaning all eight completion bits are HIGH together.
- The receiver then checks rs_error and rs_valid to evaluate the reported decoding status.

Meaning: All eight RS decoders must finish before the receiver can complete the recovery process. The exact interpretation of error and validity status depends on the implementation.

Stage 6 — Original data is recovered

Signals: rx_inst/data_internal, rx_data, original_data, rx_done

- The receiver takes the data portion of each decoded RS codeword and combines the eight blocks.
- rx_data contains the recovered data.
- The testbench compares rx_data with original_data.
- When the recovery process finishes, rx_done indicates completion.

Your earlier simulation transcript reported:

PASS: depth-8 receiver recovered original data after single-bit error

This means the testbench's comparison succeeded for that run.

➔ Viterbi decoding reverses the convolutional coding, deinterleaving restores the RS codeword arrangement, and RS decoding performs the final error correction before the original data is checked.

---

1. Main testbench + transmitter signals

Copy these into the QuestaSim Transcript:

add wave sim:/tb_full_i8/clk

add wave sim:/tb_full_i8/reset

add wave sim:/tb_full_i8/start

add wave sim:/tb_full_i8/tx_done

add wave sim:/tb_full_i8/tx_inst/state

add wave sim:/tb_full_i8/tx_inst/rs_start

add wave sim:/tb_full_i8/tx_inst/rs_inst/busy_i

add wave sim:/tb_full_i8/tx_inst/rs_inst/done_i

add wave sim:/tb_full_i8/tx_inst/code_index

add wave sim:/tb_full_i8/tx_inst/symbol_index

add wave sim:/tb_full_i8/tx_inst/codeword_select

add wave sim:/tb_full_i8/tx_inst/interleaved

add wave sim:/tb_full_i8/tx_inst/bit_index

add wave sim:/tb_full_i8/tx_inst/cc_data

add wave sim:/tb_full_i8/tx_inst/cc_valid

add wave sim:/tb_full_i8/tx_inst/cc_inst/code_out

add wave sim:/tb_full_i8/tx_inst/cc_valid_out

add wave sim:/tb_full_i8/tx_inst/cc_count

add wave sim:/tb_full_i8/tx_bits(100)

add wave sim:/tb_full_i8/channel_bits(100)

2. Receiver signals

add wave sim:/tb_full_i8/rx_start

add wave sim:/tb_full_i8/rx_inst/state

add wave sim:/tb_full_i8/rx_inst/vit_start

add wave sim:/tb_full_i8/rx_inst/vit_done

add wave sim:/tb_full_i8/rx_inst/rs_start

add wave sim:/tb_full_i8/rx_inst/rs_busy

add wave sim:/tb_full_i8/rx_inst/decoded_bits

add wave sim:/tb_full_i8/rx_inst/deinterleaved_bits

add wave sim:/tb_full_i8/rx_data

add wave sim:/tb_full_i8/original_data

add wave sim:/tb_full_i8/rx_done

add wave sim:/tb_full_i8/rx_inst/rs_error

add wave sim:/tb_full_i8/rx_inst/rs_valid

add wave sim:/tb_full_i8/rx_inst/rs_done

add wave sim:/tb_full_i8/rx_inst/data_internal

add wave sim:/tb_full_i8/rx_inst/interleaved

3. Internal Viterbi decoder signals

add wave sim:/tb_full_i8/rx_inst/vit_inst/state

add wave sim:/tb_full_i8/rx_inst/vit_inst/busy

add wave sim:/tb_full_i8/rx_inst/vit_inst/step

add wave sim:/tb_full_i8/rx_inst/vit_inst/trace_step

add wave sim:/tb_full_i8/rx_inst/vit_inst/rx_store

4. What the important values meant

Signal	**What we observed / understood**
tx_inst/state	Transmitter FSM; screenshot showed IDLE.
code_index	7 in the screenshot — the eighth codeword, using zero-based indexing.
symbol_index	FE in hexadecimal, i.e. 254.
codeword_select	0 — selecting the first codeword in zero-based indexing.
bit_index	3FC0 in hexadecimal, i.e. 16,320.
cc_count	3FC0 in hexadecimal, i.e. 16,320 output pairs counted.
tx_bits(100)	0 in the screenshot.
channel_bits(100)	1 in the screenshot — the selected bit was flipped.

rx_inst/state	COMPLETE in the receiver screenshot.
rs_busy	00 — all eight RS decoder busy bits were low.
rs_error	00 — no error flags were asserted in that displayed state.
rs_valid	FF — all eight validity bits were high.
rs_done	00 at that particular cursor position; the completion pulse/status had returned low.
vit_inst/state	IDLE after Viterbi processing.
vit_inst/busy	0 — Viterbi was not busy at that cursor position.
vit_inst/step	3FBF in hexadecimal, i.e. 16,319.
vit_inst/trace_step	1 — the displayed traceback step.

Important, bro: these are the values at the cursor positions in your screenshots, not necessarily the values throughout the simulation. After adding the signals, run run -all and type wave zoom full to inspect the complete timeline.

Step 1: Prepare your project files

You said all your VHDL code is saved in one text file.

Before going to the lab, make sure your code is accessible through an approved method. If your code is in one large text file, separate it into individual .vhd files matching the design units, unless your combined file is already valid and compiles correctly.

Our expected files were:

rs_reference_encoder.vhd

design.vhd

viterbi_frame_decoder.vhd

deinterleaver_i8.vhd

rs_decoder.vhd

tx_chain_i8.vhd

rx_chain_i8.vhd

tb_full_i8.vhd

Confirm the filenames against your actual code. Don't depend on your personal computer's old folder path being available at URSC.

Step 2: Open QuestaSim

1. Open QuestaSim 10.7c.
2. Open the Transcript window.
3. Create the simulation library by entering:

vlib work

vmap work work

If the work library already exists, you may not need to recreate it.

Step 3: Compile all VHDL files

Compile the files in dependency order, one at a time:

vcom -2008 rs_reference_encoder.vhd

vcom -2008 design.vhd

vcom -2008 viterbi_frame_decoder.vhd

vcom -2008 deinterleaver_i8.vhd

vcom -2008 rs_decoder.vhd

vcom -2008 tx_chain_i8.vhd

vcom -2008 rx_chain_i8.vhd

vcom -2008 tb_full_i8.vhd

Wait for each command to finish. If you get a compilation error, stop and fix it before compiling the next file.

These commands assume the files are in the current working directory. If not, navigate to the folder where the approved copy of your files is stored.

Step 4: Start the simulation

Enter:

vsim work.tb_full_i8

This loads your top-level testbench. It does not run the simulation yet.

Step 5: Add all the signals we studied

A. Main testbench signals

add wave sim:/tb_full_i8/clk

add wave sim:/tb_full_i8/reset

add wave sim:/tb_full_i8/start

add wave sim:/tb_full_i8/tx_done

add wave sim:/tb_full_i8/rx_start

add wave sim:/tb_full_i8/rx_done

add wave sim:/tb_full_i8/tx_bits(100)

add wave sim:/tb_full_i8/channel_bits(100)

add wave sim:/tb_full_i8/rx_data

add wave sim:/tb_full_i8/original_data

B. Transmitter signals

add wave sim:/tb_full_i8/tx_inst/state

add wave sim:/tb_full_i8/tx_inst/rs_start

add wave sim:/tb_full_i8/tx_inst/rs_inst/busy_i

add wave sim:/tb_full_i8/tx_inst/rs_inst/done_i

add wave sim:/tb_full_i8/tx_inst/code_index

add wave sim:/tb_full_i8/tx_inst/symbol_index

add wave sim:/tb_full_i8/tx_inst/codeword_select

add wave sim:/tb_full_i8/tx_inst/interleaved

add wave sim:/tb_full_i8/tx_inst/bit_index

add wave sim:/tb_full_i8/tx_inst/cc_data

add wave sim:/tb_full_i8/tx_inst/cc_valid

add wave sim:/tb_full_i8/tx_inst/cc_inst/code_out

add wave sim:/tb_full_i8/tx_inst/cc_valid_out

add wave sim:/tb_full_i8/tx_inst/cc_count

C. Receiver signals

add wave sim:/tb_full_i8/rx_inst/state

add wave sim:/tb_full_i8/rx_inst/vit_start

add wave sim:/tb_full_i8/rx_inst/vit_done

add wave sim:/tb_full_i8/rx_inst/decoded_bits

add wave sim:/tb_full_i8/rx_inst/deinterleaved_bits

add wave sim:/tb_full_i8/rx_inst/rs_start

add wave sim:/tb_full_i8/rx_inst/rs_busy

add wave sim:/tb_full_i8/rx_inst/rs_done

add wave sim:/tb_full_i8/rx_inst/rs_error

add wave sim:/tb_full_i8/rx_inst/rs_valid

add wave sim:/tb_full_i8/rx_inst/data_internal

add wave sim:/tb_full_i8/rx_inst/interleaved

D. Internal Viterbi signals

add wave sim:/tb_full_i8/rx_inst/vit_inst/state

add wave sim:/tb_full_i8/rx_inst/vit_inst/busy

add wave sim:/tb_full_i8/rx_inst/vit_inst/step

add wave sim:/tb_full_i8/rx_inst/vit_inst/trace_step

add wave sim:/tb_full_i8/rx_inst/vit_inst/rx_store

If a signal produces a “No objects found” error, don't panic. Some internal signals may not be visible in that compiled design. Continue with the signals that are available.

Step 6: Run the complete simulation

Enter:

run -all

Wait for the testbench to finish. Then enter:

wave zoom full

This displays the full simulation timeline.

Step 7: Understand the waveform from beginning to end

1. Reset and start

reset goes high to initialize the design, then low. start triggers the transmitter.

2. RS encoding

The transmitter processes eight RS codewords. Watch rs_start, busy_i, done_i and code_index.

3. Interleaving and CC encoding

Watch symbol_index, codeword_select, interleaved, cc_data, cc_valid, code_out and cc_count.

4. Channel error

Compare tx_bits(100) and channel_bits(100). In your screenshot, the values were 0 and 1, respectively, showing the injected bit flip.

5. Viterbi decoding

The Viterbi state moves through CALCULATE, TRACEBACK, FINISH and back to IDLE. Watch busy, step and trace_step.

6. Deinterleaving and RS decoding

Watch deinterleaved_bits, rs_busy, rs_done, rs_error and rs_valid.

7. Final verification

Compare rx_data against original_data and check the Transcript for the PASS message.

Step 8: Confirm the expected result

Our previous successful run produced:

Starting depth-8 RS + CC single-bit error test

Depth-8 transmitter completed

Single-bit channel error injected

PASS: depth-8 receiver recovered original data after single-bit error

The final rx_data should match original_data for the testbench assertion to pass.

One important thing, bro: the screenshots we studied showed specific values at particular cursor positions. Your new run may show different intermediate values depending on where the cursor is placed, so focus on the transitions and the final PASS result.